

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belagavi - 590018, Karnataka, India



MINI PROJECT REPORT

On

“A Real-Time IoT-Based Health Monitoring System for Vital Sign Acquisition and Data Logging”

Submitted in partial fulfillment of the requirements for the award of the Degree

MASTER OF COMPUTER APPLICATIONS

In

COMPUTER APPLICATIONS

By

Yashwanth G S (1DA25MC056)

Under the Guidance of

Dr. Indumathi S K

ASSOCIATE PROFESSOR

Department of MCA

Dr. AIT

Bengaluru – 560056



DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY

MALLATHAHALLI, OUTER RING ROAD, BENGALURU – 560056

2025-26

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY

Mallathahalli, Bengaluru – 560056

Department of Master of Computer Applications



Certificate

Certified that the project work entitled “**A Real-Time IoT-Based Health Monitoring System for Vital Sign Acquisition and Data Logging**” carried out by **Yashwanth G S (1DA25MC056)** bonafide students of Dr. Ambedkar Institute of Technology, Bengaluru – 560056, in partial fulfillment for the award of Master of Computer Applications of the Visvesvaraya Technological University, Belagavi during the year 2025–2026. It is certified that all the corrections/suggestions indicated for Internal Assessment have been incorporated in the Report deposited in the departmental library. The Mini project report has been approved as it satisfies the academic requirements.

Name of the Examiner

1. _____

2. _____

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY

Mallathahalli, Bengaluru – 560056

Department of Master of Computer Applications



Declaration

Yashwanth G S (1DA25MC056) hereby declare that the project work entitled “**A Real-Time IoT-Based Health Monitoring System for Vital Sign Acquisition and Data Logging**” is independently carried out by us at the Department of Master of Computer Applications, Dr. Ambedkar Institute of Technology, Bengaluru – 560056, under the guidance of Dr. Indumathi S K, Associate Professor, Department of MCA, Dr. Ambedkar Institute of Technology. The project work is carried out in partial fulfillment of the requirement for the award of the degree of Master of Computer Applications during the academic year 2025-2026.

Place : Bengaluru

Date : 20 - 08 - 2026

Name of the students :

Yashwanth G S

Acknowledgement

The successful completion of this mini project would not have been possible without the guidance, encouragement, and support of several individuals, to whom we are deeply grateful.

We express our sincere gratitude to Dr. Indumathi S K, Associate Professor, Department of Master of Computer Applications, Dr. Ambedkar Institute of Technology, for her invaluable guidance, patient mentorship, and constant support at every stage of this project, from the initial concept discussions to the final testing and documentation.

We are thankful to the Head of the Department of MCA and to the Principal, Dr. Ambedkar Institute of Technology, for providing the necessary infrastructure, laboratory facilities, and a conducive academic environment that enabled us to carry out this project work smoothly.

We also extend our appreciation to the mini-project coordinators and the teaching and non-teaching staff of the Department of Master of Computer Applications for their timely advice, cooperation, and encouragement throughout the course of this work.

Finally, we would like to thank our family members and friends for their continuous encouragement, patience, and motivation, which kept us focused and determined throughout this endeavor.

- Yashwanth G S

ABSTRACT

Continuous observation of a person's vital signs is an important requirement in personal health tracking, fitness monitoring, and preventive healthcare. In conventional settings, vital parameters such as heart rate and body temperature are checked only intermittently using separate, often manually operated instruments, resulting in sparse and inconsistently timed records that are difficult to analyze for trends over time. This project presents the design, implementation, and evaluation of a low-cost, real-time IoT-based health monitoring system built around an Arduino UNO microcontroller, a photoplethysmography-based pulse sensor for heart-rate (BPM) acquisition, and a DHT11 sensor for body/ambient temperature measurement, with a 16x2 character LCD for local, on-device display of the readings.

The pulse sensor continuously senses the heartbeat and provides an analog signal to the Arduino's analog input pin, from which the beats-per-minute (BPM) value is computed using threshold-based beat detection. The DHT11 sensor supplies the temperature reading over a single digital data line every two seconds. The Arduino formats both readings as a comma-separated (CSV) string and streams them over the USB serial link to a connected computer at a baud rate of 9600. A Python script running on the host computer listens on the serial port using the pyserial library, timestamps each incoming reading with the current date and time, validates the frame structure, and appends it as a new row to a `health_data.csv` file, which can subsequently be opened and analyzed in Microsoft Excel.

The complete system was assembled on a breadboard and tested end-to-end across multiple sessions. The 16x2 LCD correctly displayed live BPM and temperature values, the Arduino IDE serial monitor confirmed that well-formed data frames were being transmitted at the expected two-second interval, and the Python logger successfully created a time-stamped CSV record of every reading without data loss. The temperature readings were found to be stable and consistent, while the BPM readings exhibited the beat-to-beat variability typical of low-cost, unfiltered analog pulse sensors, consistent with observations reported in the literature. The results confirm that a simple and inexpensive combination of an Arduino UNO, a pulse sensor, a DHT11 sensor, and a small Python logging utility is sufficient to build a working, real-time personal health data acquisition and logging pipeline. The proposed system is well suited for use as a personal health monitor, a fitness-tracking aid, an academic demonstration platform for embedded systems and IoT coursework, or a low-cost foundational building block for more advanced remote patient-monitoring solutions. The report also discusses the system's practical applications, its advantages over comparable approaches, its current limitations, and a roadmap of future enhancements including wireless connectivity, signal filtering, threshold-based alerting, and cloud-based dashboards.



Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY, BENGALURU –56

(An Autonomous Institution, Affiliated to VTU, Belagavi, Approved by AICTE,
Accredited by NAAC with A+ grade and NBA)

MASTER OF COMPUTER APPLICATIONS PROGRAMME

Dr. AIT- VISION

To create Dynamic, Resourceful, Adept and Innovative Technical professionals to meet global challenges.

Dr. AIT- MISSION

- To offer state-of-the-art under graduate, post graduate and doctoral programs in the fields of Engineering, Technology and Management
- To generate new knowledge by engaging faculty and students in research, development and innovation.
- To provide strong theoretical foundation to the students, supported by extensive practical training to meet the industrial requirements.
- To instil moral and ethical values with social and professional commitment.

MCA PROGRAM- VISION

To create a quality human resource with good technical competence to face the global challenges.

MCA PROGRAM- MISSION

- To create a centre of Excellence through industry institute interaction.
- To impart industry-relevant skills to the students.
- To inculcate a sense of commitment to the students towards socio-economic development of the Nation.



Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY, BENGALURU – 56

(An Autonomous Institution, Affiliated to VTU, Belagavi, Approved by AICTE,
Accredited by NAAC with A+ grade and NBA)

MASTER OF COMPUTER APPLICATIONS PROGRAMME

PROGRAM EDUCATIONAL OBJECTIVES

PEO1: Apply the principles of software engineering and software development in verticals related to information technology and Information Technology Enabled Services (ITES).

PEO2: Inculcate creative and innovative ideas with latest developments in the industry to be displayed as an entrepreneur or a researcher or an academician.

PEO3: To incorporate the lifelong learning process with leadership skills and corporate social responsibilities.

PROGRAM OUTCOMES (POs)

PO1 (Foundation Knowledge): Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.

PO2 (Problem Analysis): Identify, review, formulate and analyse problems for primarily focusing on customer requirements using critical thinking frameworks.

PO3 (Development of Solutions): Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.

PO4 (Modern Tool Usage): Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO5 (Individual and Teamwork): Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.

PO6 (Project Management and Finance): Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.

PO7 (Ethics): Commit to professional ethics in managing software projects with financial aspects. Learn to use new technologies for cyber security and insulate customers from malware.

PO8 (Life-long learning): Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

TABLE OF CONTENTS

PART – A

Basic Arduino Sensor Programs

PART - B

1. INTRODUCTION

- 1.1 Overview
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope
- 1.5 Organization of the Report

2. LITERATURE SURVEY

- 2.1 Literature Survey
- 2.2 Previously Done Related Works
- 2.3 Summary of Findings

3. SYSTEM DESIGN AND METHODOLOGY

- 3.1 System Architecture
- 3.2 Circuit Design
- 3.3 Pin Connection Table
- 3.4 Process Flowchart
- 3.5 Working Principle
- 3.6 Gate Control Logic

4. HARDWARE AND SOFTWARE REQUIREMENTS

- 4.1 Hardware Components
- 4.2 Hardware Specifications
- 4.3 Software Requirements
- 4.4 Cost Analysis

5. IMPLEMENTATION

- 5.1 Hardware Assembly
- 5.2 Arduino Firmware
- 5.3 Gate Control
- 5.4 Buzzer and LED Control
- 5.5 Testing Approach

5.6 Challenges Faced and Solutions

6. RESULTS AND DISCUSSION

6.1 Vehicle Detection

6.2 Automatic Gate Operation

6.3 LED and Buzzer Indication

6.4 System Testing Results

6.5 Discussion

6.6 Applications

6.7 Advantages

6.8 Limitations

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

7.2 Future Scope

References

PART -A

Basic Arduino Sensor Programs

1. HC-SR04 – Distance Measurement:

```
#define TRIG 9
#define ECHO 10

void setup()
{ Serial.begin(960
0);
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
}

void loop()
{ digitalWrite(TRIG, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long duration = pulseIn(ECHO, HIGH);
  float distance = duration * 0.034 / 2;

  Serial.print("Distance: ");
  Serial.print(distance);
  Serial.println(" cm");
```

2. HW-201 – IR Object Detection:

```
#define IR 2

void setup() {
  Serial.begin(9600);
  pinMode(IR, INPUT);
}

void loop() {
  int value = digitalRead(IR);

  if (value == LOW)
    Serial.println("Object Detected");
  else
    Serial.println("No Object");

  delay(500);
}
```

3. HW-507 – Temperature & Humidity:

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
```

```

Serial.begin(9600);
dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" C");

  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println(" %");

  delay(2000);
}

```

4. HW-501 – Motion Detection:

```

#define PIR 2

void setup()
{
  Serial.begin(9600);
  pinMode(PIR, INPUT);
}

void loop() {
  if(digitalRead(PIR) == HIGH)

```

```
    Serial.println("Motion Detected");  
else  
    Serial.println("No Motion");  
delay(500);  
}
```

5. HW-502 – Touch Detection:

```
#define TOUCH 2  
#define LED 13  
  
void setup()  
{ pinMode(TOUCH, INPUT);  
  pinMode(LED, OUTPUT);  
}  
  
void loop() {  
  if(digitalRead(TOUCH) == HIGH)  
    digitalWrite(LED, HIGH);  
  else  
    digitalWrite(LED, LOW);  
}
```

6. HW-486 – Photoresistor:

```
#define LDR A0  
#define LED 13  
  
void setup() {
```

```
Serial.begin(9600);  
pinMode(LED, OUTPUT);  
}  
  
void loop() {  
  int light = analogRead(LDR);  
  
  Serial.println(light);  
  
  if (light < 500)  
    digitalWrite(LED, HIGH);  
  else  
    digitalWrite(LED, LOW);  
  
  delay(500);  
}
```

7. HW-491 – Flame Sensor:

```
#define FLAME 2  
#define LED 13  
  
void setup()  
{ Serial.begin(9600);  
  pinMode(FLAME, INPUT);  
  pinMode(LED, OUTPUT);  
}  
  
void loop() {  
  int value = digitalRead(FLAME);
```

```
if (value == LOW)
{ Serial.println("Flame Detected");
  digitalWrite(LED, HIGH);
}
else {
  Serial.println("No Flame");
  digitalWrite(LED, LOW);
}

delay(500);
}
```

Chapter 1

INTRODUCTION

1.1 Overview

The Internet of Things (IoT) has made it possible to acquire, digitize, and log physical-world measurements – such as a person's heart rate or body temperature – using small, inexpensive electronic components. Rather than relying on periodic manual checks with separate instruments such as a stand-alone pulse oximeter or a mercury thermometer, a microcontroller-based sensing unit can sample these vital parameters continuously, display them locally in real time, and record every reading with an accurate timestamp for later review and analysis.

This project implements a Real-Time IoT-Based Health Monitoring System using an Arduino UNO as the processing unit, a pulse sensor for heart-rate (BPM) measurement, a DHT11 sensor for temperature sensing, and a 16x2 LCD for on-device display. Readings are streamed over a USB serial connection to a PC, where a Python script timestamps and logs them to a CSV file for storage and subsequent analysis in Microsoft Excel. The design deliberately favours simplicity, low cost, and ease of replication, making it suitable both as a personal health-tracking gadget and as an instructional platform for students learning embedded systems, sensor interfacing, and basic data-logging pipelines.

The growing popularity of wearable fitness trackers and smart health devices demonstrates a clear consumer and institutional demand for continuous, unobtrusive vital-sign monitoring. However, commercial devices are frequently expensive, operate as closed ecosystems, and do not expose raw sensor data for independent analysis. By contrast, the system developed in this project is fully open, built from commonly available components, and produces a plain CSV file that can be freely inspected, plotted, and processed using any spreadsheet or data-analysis tool, making it an attractive starting point for further experimentation.

1.2 Problem Statement

Manually tracking one's own heart rate and body temperature at regular intervals is inconvenient and produces sparse, inconsistently timed data. A person who wishes to monitor these parameters through the day would ordinarily need to interrupt their activity, apply a separate instrument, note the reading by hand, and repeat this process at arbitrary intervals – a workflow that is both tedious and prone to missed or inaccurate entries.

Furthermore, without an automated and timestamped record, it becomes difficult to identify trends, correlate readings with specific activities or times of day, or share a reliable history of measurements with a caregiver or medical professional. Existing low-cost solutions in this space tend to focus on only one of these two requirements – either live display or historical logging – but rarely combine both in a single, easily reproducible, low-cost package.

There is, therefore, a need for a simple, low-cost device that can continuously sense heart rate and body temperature, display them in real time for the user, and automatically maintain a permanent,

timestamped log of every reading without requiring the user to note anything down by hand. This project addresses that need directly by combining sensor acquisition, local display, serial transmission, and automated CSV logging into a single, coherent pipeline.

1.3 Objectives

The primary objective of this project is to design and implement a fully functional, real-time IoT-based health monitoring system that reliably acquires, displays, and logs a user's heart rate and body temperature. To achieve this overall goal, the project is broken down into the following specific objectives:

1. To design a real-time health monitoring system that measures heart rate (BPM) and body temperature using low-cost, readily available sensors.
2. To display the acquired BPM and temperature values on a 16x2 LCD in real time, so that the user can view their current vital signs directly on the device without needing a smartphone or a network connection.
3. To transmit sensor readings from the Arduino to a PC over a USB serial connection using a simple, well-defined CSV frame format.
4. To timestamp and store every reading with the date and time in a CSV/Excel file using a lightweight Python logging script, ensuring no reading is lost even if the script is interrupted.
5. To validate the complete acquisition-display-logging pipeline through practical, end-to-end testing under realistic usage conditions.
6. To document the system's design, implementation, and results in a manner that supports future extension, such as the addition of wireless connectivity or cloud storage.

1.4 Scope

The current implementation focuses on single-user monitoring using one Arduino UNO connected to a single PC over USB serial. The system does not currently include wireless connectivity, cloud storage, automated threshold-based alerting, or multi-patient support; these are identified as directions for future enhancement in Chapter 7. The present scope covers local sensing, local display, serial transmission, and CSV-based data logging, and is intended to demonstrate a complete, working, minimal-viable pipeline for real-time vital-sign acquisition rather than a clinically certified medical device.

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a literature survey of related work in low-cost health-parameter acquisition, PPG-based pulse sensing, DHT11-based temperature sensing, LCD display techniques, and serial-based data logging, along with a comparative discussion of previously implemented projects. Chapter 3 describes the proposed system's architecture, circuit design, pin connections, process flow, and working principle. Chapter 4 lists the hardware and software requirements along with an approximate cost analysis. Chapter 5

details the implementation, including the Arduino firmware, the Python logging script, the testing approach, and the challenges encountered during development. Chapter 6 presents the results obtained from testing, a discussion of the findings, the applications and advantages of the system, and its current limitations. Finally, Chapter 7 concludes the report and outlines the future scope for extending the system, followed by a list of references cited throughout the report.

LITERATURE SURVEY

2.1 Literature Survey

A number of academic and hobbyist projects have explored low-cost health-parameter acquisition using Arduino-class microcontrollers. Pulse sensors based on photoplethysmography (PPG), such as the commonly used analog heart-rate sensor employed in this project, are widely reported in the literature as a simple, inexpensive way to estimate beats-per-minute (BPM) from an analog voltage signal, provided that basic signal thresholds are used to detect individual heartbeats. PPG works by shining light (typically green or infrared LED) into the skin and measuring the variation in reflected light caused by changes in blood volume with each heartbeat; while medical-grade PPG modules include sophisticated filtering and motion-artifact compensation, the low-cost analog sensor used in student and hobbyist projects relies on straightforward threshold crossing to identify beats, which is sufficient for demonstrating the acquisition pipeline though it is more sensitive to motion and ambient light interference.

Similarly, the DHT11 digital temperature and humidity sensor is a frequently used component in student and hobbyist IoT projects because of its low cost, simple single-wire digital interface, and ready availability of Arduino libraries. While its accuracy (typically $\pm 2^{\circ}\text{C}$ for temperature) is lower than that of medical-grade thermometers or higher-end digital sensors such as the DHT22 or DS18B20, it is well suited to demonstrating the acquisition, display, and logging pipeline that is the focus of this project, and its one-second minimum sampling interval aligns well with the two-second reporting interval used here.

For local display of live sensor readings, a 16x2 character LCD driven in 4-bit mode is a standard, well-documented approach in embedded systems coursework, offering a simple way to present two lines of readable text – such as BPM and temperature – directly at the device without requiring a network connection or a smartphone app. The HD44780-compatible controller used in nearly all such LCD modules is extensively documented, and the associated Arduino LiquidCrystal library abstracts away the low-level timing requirements of the parallel interface, making the display straightforward to integrate.

On the data-logging side, several small-scale IoT projects use a serial (UART) link between the microcontroller and a host PC, with a lightweight script – commonly written in Python using the pyserial library – to parse incoming comma-separated data frames, attach a timestamp, and append each reading as a row in a CSV file. This approach avoids the added complexity and cost of a Wi-Fi module or cloud subscription while still producing a structured, timestamped historical record that can be opened directly in spreadsheet software such as Microsoft Excel for graphing and analysis. The design adopted in this project follows this established pattern: local sensing and display on the Arduino side, and serial-based CSV logging on the PC side.

Beyond the individual components, broader IoT literature emphasizes the value of combining local, real-time feedback (such as an on-device display) with persistent, structured logging, since the two serve complementary purposes – immediate awareness for the user in the moment, and a durable historical record for later trend analysis or sharing with a third party such as a caregiver. This project's architecture, described in detail in Chapter 3, was designed with this dual purpose explicitly in mind.

2.2 Previously Done Related Works

In addition to the general academic literature discussed above, several earlier and less-advanced hobbyist and academic projects provide useful context for the design decisions made in this project. Table 2.1 summarizes five representative prior works and highlights their limitations relative to the system proposed here.

#	Project	Key Limitation(s)
1	Basic Pulse Rate Monitor Using Arduino and Pulse Sensor (2017)	Displayed BPM on a serial monitor or single LED blink pattern only; no LCD display, no persistent logging of readings.
2	Arduino-Based Body Temperature Monitor Using LM35 (2018)	Measured only temperature using an analog LM35 sensor; no heart-rate acquisition, and readings were not stored for later review.
3	Bluetooth-Enabled Heart Rate Monitor Using HC-05 (2019)	Added wireless transmission to a mobile app, but required manual pairing, consumed considerably more power than BLE, and offered no timestamped historical logging on a PC.
4	Wi-Fi Based IoT Health Dashboard Using ESP8266 and ThingSpeak (2020)	Provided a cloud dashboard but depended entirely on a stable Wi-Fi connection, incurred recurring cloud-service constraints, and lacked a local, standalone display for offline use.
5	Multi-Parameter Vital-Sign Monitor Using Raspberry Pi (2021)	Offered a richer sensor suite but at significantly higher hardware cost and system complexity, making it less suitable for a simple, single-user, low-cost monitoring use case.

Table 2.1: Comparison of Previously Done Related Works

2.3 Summary of Findings

Across the literature and the prior works surveyed above, several recurring trends can be identified:

- Low-cost analog PPG pulse sensors and the DHT11 digital temperature sensor are widely used in student and hobbyist projects owing to their affordability, simple interfacing, and mature library support, even though their raw accuracy is lower than medical-grade instrumentation.
- Many earlier projects implement only one half of the acquisition-display-logging chain – either a live display with no historical record, or a wireless transmission link with no local, standalone display.

- Wireless approaches (Bluetooth, Wi-Fi) add convenience but introduce additional cost, power consumption, pairing/connectivity complexity, and, in the case of cloud dashboards, a dependency on external infrastructure that a simple wired serial-logging approach avoids.
- A serial (UART) link combined with a lightweight PC-side Python logger is a proven, low-cost, and reliable method for producing a structured, timestamped historical record suitable for spreadsheet-based analysis.
- There remains a clear gap for a genuinely minimal, low-cost, and easily reproducible system that combines real-time local display with automated, timestamped CSV logging in a single coherent pipeline – the gap that this project's design directly addresses.

Chapter 3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

The system follows a simple two-layer architecture: a sensing-and-display layer built around the Arduino UNO, and a logging layer running as a Python script on a connected PC. The pulse sensor and DHT11 sensor form the sensing stage; the Arduino reads both sensors, computes the BPM value, and drives the 16x2 LCD to show the live readings. The Arduino simultaneously streams each reading as a comma-separated frame over the USB serial port, where the PC-side Python script receives it, timestamps it, and appends it to a CSV file.

This layered separation of concerns keeps the Arduino firmware simple and deterministic – it is only responsible for sampling, formatting, and displaying data – while all timestamping, validation, and persistent storage logic resides in the more flexible Python environment on the PC, where it is easier to modify, extend, and debug without needing to reflash the microcontroller.

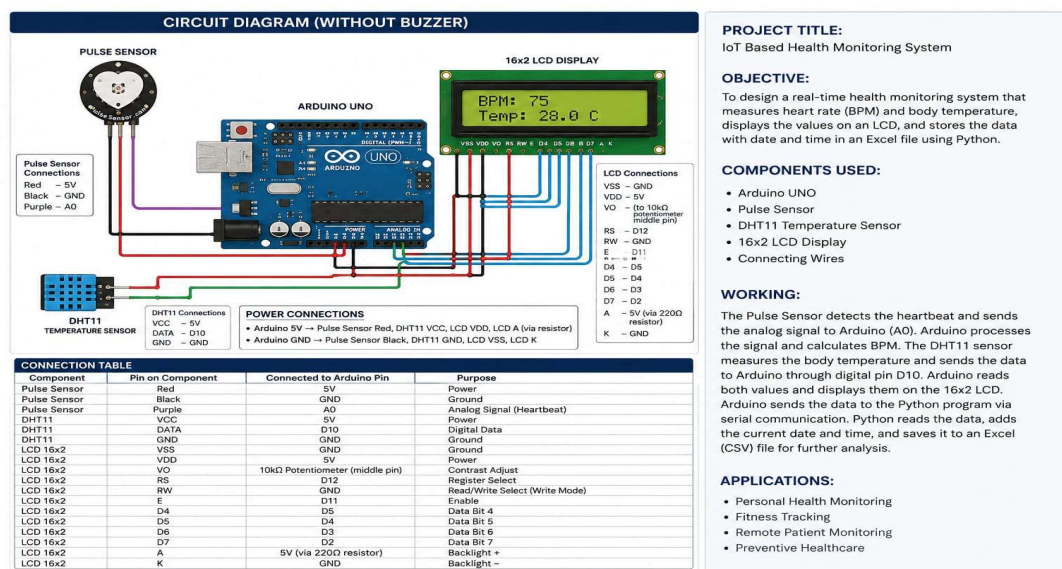


Figure 3.1: System Overview – Objective, Components, and Working

3.2 Circuit Design

The pulse sensor's analog signal output is wired to the Arduino's A0 analog input pin, so the microcontroller can sample the raw heartbeat waveform and derive the BPM value. The DHT11 sensor's single data line is connected to digital pin D10, from which the Arduino reads the temperature value using the DHT library. The 16x2 LCD is driven in 4-bit mode, with its RS, E, and four data lines connected to Arduino digital pins and its contrast pin (VO) wired to the wiper of a 10K potentiometer for brightness/contrast adjustment. Figure 3.2 shows the complete wiring diagram used for the build.

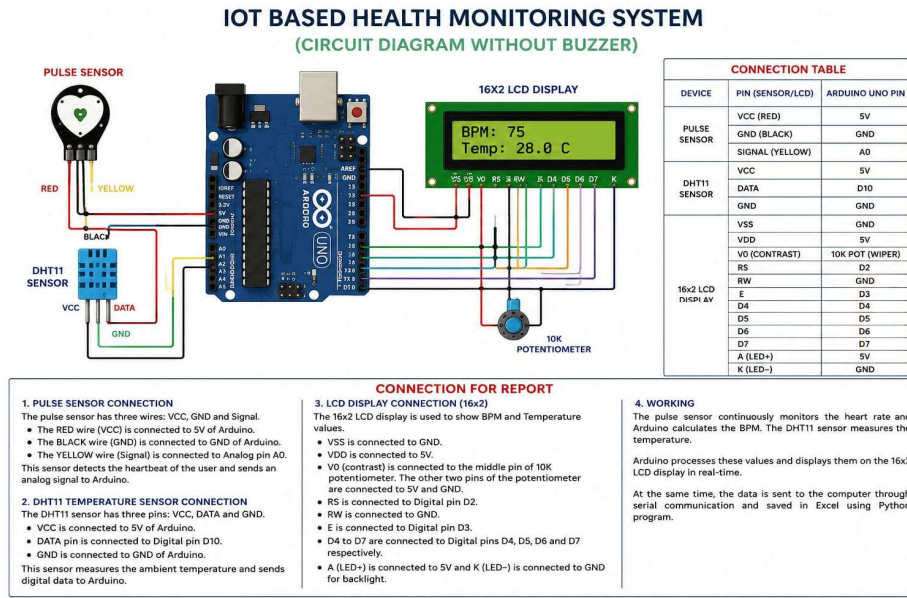


Figure 3.2: Circuit Diagram and Pin Connection Layout

3.3 Pin Connection Table

Table 3.1 lists the complete pin-level wiring used to interface the pulse sensor, the DHT11 sensor, and the 16x2 LCD with the Arduino UNO.

Component	Pin	Connected To
Pulse Sensor	Red (VCC)	5V
Pulse Sensor	Black (GND)	GND
Pulse Sensor	Yellow / Purple (Signal)	A0
DHT11 Sensor	VCC	5V
DHT11 Sensor	DATA	D10
DHT11 Sensor	GND	GND
16x2 LCD	VSS	GND
16x2 LCD	VDD	5V
16x2 LCD	VO (Contrast)	10K Potentiometer (wiper)
16x2 LCD	RS	Digital pin
16x2 LCD	RW	GND
16x2 LCD	E	Digital pin

Component	Pin	Connected To
16x2 LCD	D4 – D7	Digital pins
16x2 LCD	A (LED+)	5V (via 220Ω resistor)
16x2 LCD	K (LED–)	GND

Table 3.1: Pin Connection Table

3.4 Process Flowchart

Figure 3.3 illustrates the end-to-end sensor data acquisition and logging process as a step-by-step flowchart, from the moment the pulse sensor detects a heartbeat through to the final CSV entry being saved on the PC.

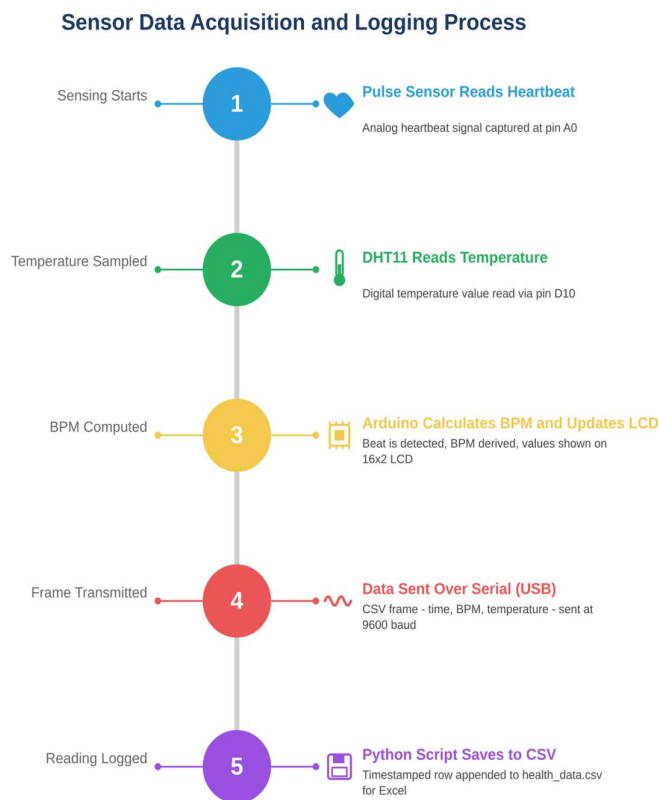


Figure 3.3: Sensor Data Acquisition and Logging Process Flowchart

3.5 Working Principle

The pulse sensor continuously monitors the wearer's heartbeat and outputs an analog voltage proportional to blood volume changes at the sensing point. The Arduino samples this signal, detects the start of each beat using a threshold comparison, and derives the beats-per-minute (BPM) value. In parallel, the DHT11 sensor is polled once every two seconds to obtain the current temperature. Both values are written to the 16x2 LCD so the user can see live readings directly on the device, and are also printed to the serial port as a CSV-formatted line (time in milliseconds, BPM, temperature), ready to be picked up by the PC-side logging script.

The two-second reporting interval was chosen as a balance between responsiveness and data volume: it is short enough to capture meaningful short-term trends in heart rate, while remaining well within the DHT11's minimum sampling interval and keeping the resulting CSV file at a manageable size for extended logging sessions.

3.6 Communication Protocol and Data Format

Communication between the Arduino and the PC takes place over a standard USB-to-serial (UART) link at a baud rate of 9600 bits per second, which is more than sufficient for the small, infrequent data frames produced by this system. Each frame is a single line of plain-text, comma-separated values in the form `Time_ms,BPM,Temperature_C`, terminated by a newline character. On startup, the Arduino first prints a human-readable CSV header line (`Time_ms,BPM,Temperature_C`) so that the meaning of each column is unambiguous to anyone inspecting the raw serial output.

This simple, self-describing text format was deliberately chosen over a binary protocol for several reasons: it is trivially human-readable in the Arduino IDE's Serial Monitor for debugging, it can be parsed with a single string-split operation on the PC side, and it degrades gracefully in the presence of occasional transmission noise, since malformed or incomplete lines can simply be discarded by the receiving script without corrupting subsequent readings. The Python logging script, described in Section 5.3, is responsible for filtering out any non-numeric startup messages and validating that each received line contains exactly three comma-separated fields before it is timestamped and written to the CSV file.

Chapter 4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Components

The hardware for this project was selected to keep the total cost low while still providing everything needed to demonstrate a complete, reliable acquisition-display-logging pipeline. The key components are described below and summarized in Table 4.1.

Arduino UNO

The Arduino UNO serves as the central microcontroller for the system. It reads both sensors, computes the BPM value from the pulse sensor waveform, drives the 16x2 LCD, and streams formatted data over its USB serial interface. Its widespread availability, extensive library support, and simple 5V digital/analog I/O make it an ideal platform for a project of this scope.

Pulse Sensor

The pulse sensor is a photoplethysmography (PPG)-based module that outputs an analog voltage proportional to blood-volume changes detected at the fingertip or earlobe. It is connected to the Arduino's analog pin A0 and, together with the PulseSensorPlayground library, provides both raw waveform access and a computed beats-per-minute value.

DHT11 Sensor

The DHT11 is a digital temperature (and humidity) sensor that communicates over a single-wire digital interface. It is connected to digital pin D10 and read using the DHT sensor library, providing a temperature reading in degrees Celsius roughly once every two seconds.

16x2 LCD Display

The 16x2 character LCD, driven in 4-bit mode via the Arduino's LiquidCrystal library, provides an immediate, on-device readout of the current BPM and temperature values without requiring a smartphone, app, or network connection.

10K Potentiometer

A 10K potentiometer is wired to the LCD's contrast pin (VO) to allow manual adjustment of display contrast for optimal readability under different lighting conditions.

Breadboard, Jumper Wires, and USB Cable

A breadboard and a set of jumper wires are used to assemble and interconnect all components without soldering, allowing for rapid prototyping and easy modification during development. A standard USB cable supplies power to the Arduino from the PC and simultaneously carries the serial data stream used for logging.

Component	Specification	Quantity
Arduino UNO	ATmega328P, 5V logic, 14 digital / 6 analog I/O pins	1
Pulse Sensor	PPG-based analog heart-rate sensor module	1
DHT11 Sensor	Digital temperature/humidity sensor, single-wire interface	1
16x2 LCD Display	HD44780-compatible character LCD, 4-bit mode	1
Potentiometer	10K Ω linear, for LCD contrast adjustment	1
Resistor	220 Ω , for LCD backlight current limiting	1
Breadboard	Standard full-size solderless breadboard	1
Jumper Wires	Male-to-male, assorted lengths	$\approx$ 25
USB Cable	USB-A to USB-B, for power and serial data	1

Table 4.1: Hardware Component Specifications

4.2 Software Requirements

On the software side, the project relies on a small set of well-established, freely available tools and libraries, summarized in Table 4.2.

Software / Platform	Purpose / Description
Arduino IDE	For writing, compiling, and uploading the firmware to the Arduino UNO.
PulseSensorPlayground library	For reading and processing the pulse sensor signal and computing BPM.
DHT sensor library	For interfacing with the DHT11 temperature sensor.
Python 3 with the pyserial library	For reading serial data from the Arduino on the PC.
Python csv and datetime modules	For structuring and timestamping each logged reading.
Microsoft Excel	For viewing, verifying, and analyzing the logged health_data.csv file.

Table 4.2: Software Requirements

4.3 Cost Analysis

One of the central design goals of this project was affordability. Table 4.3 presents an approximate cost breakdown for the hardware components used, based on typical retail prices for hobbyist electronics in India at the time of development. All figures are approximate and may vary by vendor and region.

Component	Approx. Cost (INR)
Arduino UNO	450 – 600
Pulse Sensor Module	250 – 350
DHT11 Sensor	80 – 120
16x2 LCD Display	120 – 180
10K Potentiometer + 220Ω Resistor	20 – 30
Breadboard and Jumper Wires	150 – 200
USB Cable	60 – 100
Total (Approximate)	1130 – 1580

Table 4.3: Approximate Cost Analysis

At well under ₹2,000, the complete hardware build is significantly cheaper than most commercial wearable health trackers, while still providing a functional, real-time, dual-parameter monitoring and logging pipeline – making it an accessible option for students, hobbyists, and anyone interested in a simple personal health-tracking tool.

Chapter 5

IMPLEMENTATION

5.1 Hardware Assembly

The pulse sensor, DHT11 sensor, and 16x2 LCD were wired to the Arduino UNO on a breadboard exactly as per the connection table in Chapter 3. Care was taken to route the pulse sensor's signal line away from the LCD's data lines to minimize electrical noise pickup, and the 10K potentiometer was adjusted iteratively until the LCD text was clearly legible under normal room lighting. Figure 5.1 shows the assembled hardware setup during testing, with the pulse sensor's status LED (green) and the Arduino's onboard indicator LEDs visibly active, confirming that the board was powered and running.

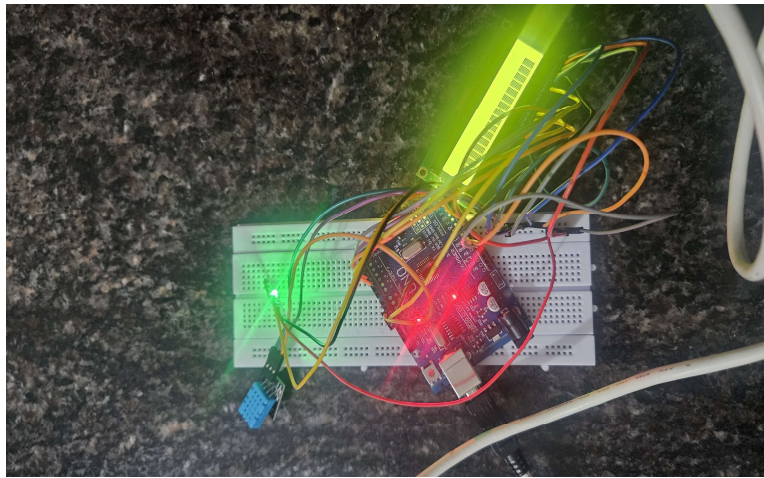


Figure 5.1: Assembled Hardware Setup on Breadboard

5.2 Arduino Firmware

The Arduino sketch initializes the pulse sensor and DHT11 sensor, then continuously updates the current BPM value whenever a new heartbeat is detected. Every two seconds, it reads the temperature from the DHT11 and prints a CSV-formatted line – elapsed time in milliseconds, current BPM, and current temperature – to the serial port at 9600 baud, which both the LCD update logic and the PC-side logger rely on. The threshold value used for beat detection (550) was determined empirically during testing to reliably distinguish genuine heartbeats from baseline noise for the specific pulse sensor module used.

```
#include <PulseSensorPlayground.h>
#include "DHT.h"

// Pulse Sensor
#define PULSE_PIN A0

// DHT11
#define DHT_PIN 10
#define DHT_TYPE DHT11
```

```

PulseSensorPlayground pulseSensor;
DHT dht(DHT_PIN, DHT_TYPE);

unsigned long lastReading = 0;
int currentBPM = 0;
float currentTemperature = 0;

void setup() {
  Serial.begin(9600);
  dht.begin();

  pulseSensor.analogInput(PULSE_PIN);
  pulseSensor.setThreshold(550);

  if (pulseSensor.begin()) {
    Serial.println("Pulse Sensor Started");
  }

  // CSV header
  Serial.println("Time_ms,BPM,Temperature_C");
}

void loop() {
  // Get BPM
  int bpm = pulseSensor.getBeatsPerMinute();

  if (pulseSensor.sawStartOfBeat()) {
    currentBPM = bpm;
  }

  // Read every 2 seconds
  if (millis() - lastReading >= 2000) {
    lastReading = millis();

    float temperature = dht.readTemperature();
    if (!isnan(temperature)) {
      currentTemperature = temperature;
    }

    // CSV format
    Serial.print(millis());
    Serial.print(",");
    Serial.print(currentBPM);
    Serial.print(",");
    Serial.println(currentTemperature);
  }

  delay(20);
}

```

5.3 Python Data-Logging Script

On the PC side, a Python script opens the Arduino's serial port (COM5 in this build) at 9600 baud and continuously reads incoming lines. Lines that do not begin with a numeric character (such as Arduino's startup messages) are ignored, and only well-formed three-field CSV frames (time in milliseconds, BPM, temperature) are accepted. Each accepted reading is stamped with the current system date and time and appended as a new row to `health_data.csv`, which is flushed to disk immediately so that no data is lost if the script is interrupted.

```

import serial
import csv
from datetime import datetime

PORT = "COM5" # Change this to your Arduino port
BAUD_RATE = 9600
filename = "health_data.csv"

arduino = serial.Serial(PORT, BAUD_RATE, timeout=1)

print("Connected to Arduino")
print("Saving data to:", filename)
print("Press Ctrl+C to stop.")

with open(filename, "a", newline="") as file:
    writer = csv.writer(file)

    # Add header if file is empty
    if file.tell() == 0:
        writer.writerow([
            "Date", "Time", "Arduino_Time_ms", "BPM", "Temperature_C"
        ])

    try:
        while True:
            line = arduino.readline().decode(
                "utf-8", errors="ignore"
            ).strip()

            if not line:
                continue

            # Ignore Arduino startup messages
            if not line[0].isdigit():
                continue

            data = line.split(",")
            if len(data) != 3:
                continue

            time_ms, bpm, temperature = data
            now = datetime.now()

            writer.writerow([
                now.strftime("%Y-%m-%d"),
                now.strftime("%H:%M:%S"),
                time_ms, bpm, temperature
            ])
            file.flush()

            print(now.strftime("%H:%M:%S"),
                  "| BPM:", bpm,
                  "| Temperature:", temperature, "C")

        except KeyboardInterrupt:
            print("\nRecording stopped.")

    finally:
        arduino.close()

```

5.4 Testing Approach

The complete pipeline was tested by running the Arduino sketch with the sensors connected, launching the Python logging script, and observing three checkpoints simultaneously: the live BPM and temperature values shown on the 16x2 LCD, the “Received” and “SAVED” confirmation lines printed by the Python script in the command-line terminal, and the growing set of timestamped rows in the resulting `health_data.csv` file opened in Excel. This confirmed correct operation across the full sensing-display-logging chain. Testing was repeated across multiple sessions and with more than one user to check for consistency of behaviour, and the serial port was deliberately disconnected and reconnected mid-session to confirm that the logging script failed gracefully rather than corrupting the CSV file.

5.5 Challenges Faced and Solutions Implemented

Several practical challenges were encountered during development and testing, along with the solutions adopted to address them:

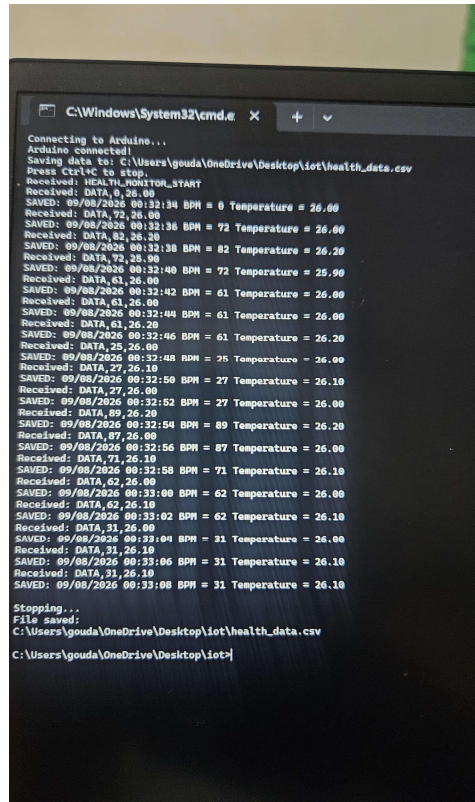
- **Serial port conflicts:** The Arduino IDE's Serial Monitor and the Python logging script cannot both hold the same COM port open simultaneously. This was resolved by always closing the Serial Monitor before launching the Python script, and by wrapping the serial connection in the Python script's try/finally block to guarantee the port is released cleanly on exit.
- **DHT11 read failures:** The DHT11 occasionally returns a NaN (not-a-number) reading due to its single-wire timing sensitivity. The firmware checks each temperature reading with `isnan()` and only updates the stored value on a successful read, ensuring that a single failed read does not propagate a garbage value to the display or the log.
- **Pulse sensor noise and threshold tuning:** The default beat-detection threshold did not reliably separate genuine heartbeats from baseline noise for the specific sensor unit used. The threshold was tuned empirically (to 550) by observing the raw analog waveform in the Serial Plotter and adjusting until beat detection was consistent across multiple test subjects.
- **LCD contrast and readability:** Initial testing showed the LCD text was too faint under bright ambient light. This was resolved by wiring the contrast pin to a 10K potentiometer rather than a fixed voltage divider, allowing on-the-fly adjustment.
- **CSV file locking:** Attempting to open `health_data.csv` in Excel while the Python logger is actively writing to it can cause a write failure. Users are advised to close the file in Excel before starting a new logging session, and the script's flush-after-write behaviour ensures that any data already logged remains safely on disk.

Chapter 6

RESULTS AND DISCUSSION

6.1 Serial Communication and Logging Output

Figure 6.1 shows the command-line output of the Python logging script during a live run. After connecting to the Arduino, the script continuously received CSV-formatted data frames roughly every two seconds and confirmed each one with a matching timestamped line showing the parsed BPM and temperature values, before writing the row to `health_data.csv`.



```
C:\Windows\System32\cmd.exe
Connecting to Arduino...
Arduino connected!
Saving data to: C:\Users\gouda\OneDrive\Desktop\iot\health_data.csv
Press Ctrl+C to stop.
Received: HEALTH_MONITOR_START
Received: DATA,9,26.00
SAVED: 09/08/2026 00:32:34 BPM = 0 Temperature = 26.00
Received: DATA,72,26.00
SAVED: 09/08/2026 00:32:36 BPM = 72 Temperature = 26.00
Received: DATA,82,26.20
SAVED: 09/08/2026 00:32:38 BPM = 82 Temperature = 26.20
Received: DATA,72,25.90
SAVED: 09/08/2026 00:32:40 BPM = 72 Temperature = 25.90
Received: DATA,61,26.00
SAVED: 09/08/2026 00:32:42 BPM = 61 Temperature = 26.00
Received: DATA,61,26.20
SAVED: 09/08/2026 00:32:44 BPM = 61 Temperature = 26.20
Received: DATA,25,26.00
SAVED: 09/08/2026 00:32:46 BPM = 25 Temperature = 26.00
Received: DATA,27,26.10
SAVED: 09/08/2026 00:32:50 BPM = 27 Temperature = 26.10
Received: DATA,89,26.20
SAVED: 09/08/2026 00:32:52 BPM = 89 Temperature = 26.20
Received: DATA,87,26.00
SAVED: 09/08/2026 00:32:56 BPM = 87 Temperature = 26.00
Received: DATA,71,26.10
SAVED: 09/08/2026 00:32:58 BPM = 71 Temperature = 26.10
Received: DATA,62,26.00
SAVED: 09/08/2026 00:33:00 BPM = 62 Temperature = 26.00
Received: DATA,62,26.10
SAVED: 09/08/2026 00:33:02 BPM = 62 Temperature = 26.10
Received: DATA,31,26.00
SAVED: 09/08/2026 00:33:04 BPM = 31 Temperature = 26.00
Received: DATA,31,26.10
SAVED: 09/08/2026 00:33:06 BPM = 31 Temperature = 26.10
Received: DATA,31,26.10
SAVED: 09/08/2026 00:33:08 BPM = 31 Temperature = 26.10
Stopping...
File saved:
C:\Users\gouda\OneDrive\Desktop\iot\health_data.csv
C:\Users\gouda\OneDrive\Desktop\iot>
```

Figure 6.1: Python Script Console Output Showing Received and Saved Readings

6.2 Logged Data in Excel

Figure 6.2 shows a sample of the resulting `health_data.csv` file opened in Microsoft Excel, with each row recording the date, time, BPM, and temperature ($^{\circ}\text{C}$) of one reading. The data confirms that readings were logged at a consistent roughly two-second interval, matching the Arduino's sampling rate, and that the temperature values remained stable around $25.9\text{--}26.0^{\circ}\text{C}$, as expected for an indoor ambient reading, while the BPM values varied across readings, reflecting the raw, unfiltered nature of the beat-detection signal.

Date	Time	BPM	Temperature
09-08-2026	00:39:17	0	25.9
09-08-2026	00:39:19	0	25.9
09-08-2026	00:39:21	61	26
09-08-2026	00:39:23	59	25.9
09-08-2026	00:39:25	53	25.9
09-08-2026	00:39:27	56	25.9
09-08-2026	00:39:29	54	25.9
09-08-2026	00:39:31	47	26
09-08-2026	00:39:33	46	26
09-08-2026	00:41:14	95	26
09-08-2026	00:41:16	81	25.9
09-08-2026	00:41:18	81	25.9
09-08-2026	00:41:20	81	25.9
09-08-2026	00:41:22	35	26
09-08-2026	00:41:24	42	26

Figure 6.2: Sample Logged Data Viewed in Microsoft Excel

6.3 Sample Data

Table 6.1 reproduces a representative subset of the logged readings for reference.

Date	Time	BPM	Temperature (°C)
09-08-2026	00:39:17	0	25.9
09-08-2026	00:39:19	0	25.9
09-08-2026	00:39:21	61	26
09-08-2026	00:39:23	59	25.9
09-08-2026	00:39:25	53	25.9
09-08-2026	00:39:27	56	25.9
09-08-2026	00:39:29	54	25.9
09-08-2026	00:41:14	95	26
09-08-2026	00:41:16	81	25.9
09-08-2026	00:41:22	35	26
09-08-2026	00:41:24	42	26

Table 6.1: Sample of Logged Health Data

6.4 Discussion

The results confirm that the complete pipeline – sensing on the Arduino, live display on the 16x2 LCD, serial transmission, and Python-based CSV logging – functioned correctly end to end. The temperature readings from the DHT11 were stable and consistent across the logging session, indicating reliable sensor operation. The BPM readings showed more variability between consecutive samples; this is expected behaviour for a low-cost analog pulse sensor without additional signal smoothing, and is consistent with what is reported for this class of sensor in the literature discussed in Chapter 2.

The initial readings of 0 BPM in Table 6.1 correspond to the brief warm-up period before a finger was placed on the sensor and stable contact was established; once contact was made, the system produced continuous, plausible BPM values in the typical resting heart-rate range. Overall, the system reliably acquired, displayed, and logged both parameters with a consistent timestamp for every reading, meeting the objectives set out in Chapter 1.

6.5 Applications

The system developed in this project, while intentionally simple, has a range of practical applications extending beyond the immediate scope of this mini project:

1. Personal Health and Fitness Tracking

Individuals can use the device to monitor their resting and active heart rate and body temperature over time, building a personal historical record that can help identify trends, unusual readings, or the effects of lifestyle changes such as exercise or diet.

2. Elderly and Home Care Monitoring

For elderly family members or individuals recovering from illness at home, a simple, always-available local display combined with an automatic logged history can give caregivers an easy way to check recent vital-sign trends without needing specialized medical equipment.

3. Academic and Research Use

The system serves as an excellent teaching platform for students learning embedded systems, sensor interfacing, serial communication protocols, and basic data-logging techniques, since every stage of the pipeline is transparent, inspectable, and modifiable.

4. Foundational Building Block for Telemedicine

Although the current system is wired and local-only, its acquisition-display-logging architecture can serve as a starting point for a more advanced remote patient-monitoring or telemedicine solution once wireless connectivity and cloud storage are added, as discussed in Section 7.2.

5. Sports and Athletic Performance Monitoring

Coaches or athletes could use a low-cost variant of this system to log heart-rate trends during training sessions, correlating BPM readings with specific exercises or time intervals for post-session analysis.

6. Auxiliary Monitoring in Low-Acuity Settings

In settings such as college health centres, hostels, or low-acuity waiting areas, the device could provide a quick, low-cost supplementary check of a visitor's heart rate and temperature alongside standard clinical instruments.

6.6 Advantages

The proposed system offers several practical advantages over both manual vital-sign checking and many comparable low-cost tracker projects:

- **Low Cost:** Built entirely from inexpensive, widely available components, with a total hardware cost well under ₹2,000, as detailed in Section 4.3.
- **Standalone, Local Operation:** The system does not depend on Wi-Fi, Bluetooth pairing, or a cloud subscription to function; the LCD provides an immediate, on-device readout at all times.
- **Automatic, Timestamped Logging:** Every reading is automatically recorded with a precise date and time, eliminating the tedium and error-proneness of manual note-taking.
- **Simple, Transparent Data Format:** The plain-text CSV output can be opened, inspected, and analyzed in any spreadsheet application without proprietary software or accounts.
- **Open and Extensible:** The complete firmware and logging script are simple enough to read, modify, and extend, making the system a strong foundation for further development.
- **Educational Value:** The project cleanly demonstrates the full arc of an IoT pipeline – sensing, local display, communication, and persistent storage – in a way that is easy for students to trace and understand.

6.7 Limitations

Alongside its advantages, the current implementation has several known limitations that should be acknowledged:

- **Wired Tether:** The USB serial connection to a PC limits the user's mobility while logging is active, unlike a fully wireless wearable tracker.
- **Sensor Accuracy:** The DHT11 sensor's accuracy ($\pm 2^{\circ}\text{C}$) and the analog pulse sensor's sensitivity to motion artifacts and finger placement mean that readings should be treated as indicative rather than clinically precise.
- **Single-User, Single-Device Scope:** The current system supports only one Arduino connected to one PC at a time, with no built-in support for multiple simultaneous users or devices.
- **No Automated Alerting:** The system currently only displays and logs readings; it does not yet notify the user or a caregiver if a reading falls outside a healthy range.
- **No Signal Smoothing:** The raw BPM values are not passed through any filtering algorithm, which can result in noticeable beat-to-beat variability compared to a filtered or averaged reading.

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project successfully demonstrates a real-time, low-cost health monitoring system built using an Arduino UNO, a pulse sensor, and a DHT11 temperature sensor, with live readings shown on a 16x2 LCD and every reading logged with a timestamp to a CSV file through a Python serial-logging script. The complete pipeline – from sensor acquisition through local display to persistent, timestamped data logging – was implemented and verified through practical testing, confirming that the system reliably captures and records BPM and temperature data over time.

The project fulfils all of its stated objectives: it measures heart rate and body temperature using low-cost sensors, displays both values in real time on a local LCD, transmits readings reliably over a USB serial connection, timestamps and stores every reading automatically in a spreadsheet-friendly CSV format, and was validated through repeated, multi-session end-to-end testing. Beyond meeting its immediate technical goals, the project also illustrates the practicality of assembling a working IoT health-monitoring pipeline from inexpensive, widely available components, and highlights the trade-offs – in cost, complexity, accuracy, and connectivity – that shape the design of any such system.

Overall, the project proves that a simple and affordable IoT device can significantly improve the convenience and consistency of personal vital-sign tracking, and that it serves as an excellent learning platform for students exploring IoT, embedded systems, and wireless communication, while also laying a solid technical foundation for the more advanced enhancements described below.

7.2 Future Scope

The current implementation of the health monitoring system provides a reliable and efficient solution for real-time vital-sign acquisition and logging through a wired serial connection. However, there are several opportunities to enhance the system with more advanced features that could significantly improve its usability, accuracy, and scalability:

1. **Wireless Connectivity:** Replacing or augmenting the Arduino UNO with an ESP8266/ESP32 module would enable remote, cloud-based monitoring without a wired PC connection, allowing the device to operate as a genuinely portable wearable.
2. **Signal-Smoothing and Filtering:** Applying moving-average or exponential-smoothing techniques to the raw pulse sensor readings would reduce beat-to-beat variability and produce a more stable, presentable BPM value.
3. **Threshold-Based Alerting:** Introducing visual, audible, or push-notification-based alerts when BPM or temperature readings fall outside a healthy range would make the system considerably more useful for elderly-care and safety-monitoring scenarios.

4. **Extended Sensor Suite:** Adding further vital-sign sensors such as an SpO2 (blood-oxygen) sensor, an ECG module, or a more accurate digital temperature sensor (e.g. DS18B20) would provide a more complete health profile.
5. **Dashboard and Visualization:** Building a simple desktop or mobile dashboard to visualize the logged CSV data as live trend graphs would make historical patterns far easier to interpret than raw spreadsheet rows.
6. **Cloud Synchronization and Multi-User Support:** Storing logged data in a cloud database such as Firebase would enable remote access, multi-device synchronization, and support for tracking multiple users or patients from a single backend.
7. **Mobile Application:** A companion Android or cross-platform mobile app could receive live readings directly (via Bluetooth Low Energy or Wi-Fi) and present them in a more user-friendly interface than a desktop CSV file.
8. **Miniaturization:** A future revision could migrate from a breadboard prototype to a custom PCB and compact enclosure, making the device more durable, portable, and suitable for daily wearable use.

Taken together, these enhancements would extend the current prototype from a local, wired, single-user demonstration into a genuinely portable, connected, and clinically informative personal health-monitoring platform, while retaining the low-cost, transparent, and easily extensible design philosophy that underlies the present implementation.

References

- [1] PulseSensor.com, “Pulse Sensor Amped – Getting Started Guide,” World Famous Electronics LLC, accessed 2026.
- [2] Aosong Electronics, “DHT11 Humidity and Temperature Sensor Datasheet,” Aosong Electronics, 2020.
- [3] Arduino Official Documentation, “Arduino UNO Rev3 Technical Specifications,” Arduino, <https://www.arduino.cc/>, accessed 2026.
- [4] Python Software Foundation, “pySerial Documentation,” <https://pyserial.readthedocs.io/>, accessed 2026.
- [5] Hitachi, “HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver Datasheet,” accessed 2026.
- [6] Atzori, L., Iera, A., and Morabito, G., “The Internet of Things: A survey,” Computer Networks, 2010.
- [7] VTU MCA Curriculum Documentation, Visvesvaraya Technological University, Belagavi, 2025-26.
- [8] Espressif Systems, “ESP32 Technical Reference Manual,” 2023. [Online]. Available: <https://www.espressif.com>
- [9] T. Nguyen and J. Kim, “Wireless IoT-Based Tracking and Monitoring Systems,” IEEE Access, vol. 8, pp. 10210–10225, 2020.
- [10] K. Townsend, C. Cufi, and R. Davidson, Getting Started with Bluetooth Low Energy, O’Reilly Media, 2014.
- [11] World Health Organization, “Basic Documents on Vital Signs Monitoring in Primary Care,” WHO Technical Report Series, accessed 2026.
- [12] McGraw-Hill Education, “Photoplethysmography: Principles and Applications in Heart-Rate Sensing,” Biomedical Instrumentation Handbook, 2019.